Modular code development - Making reusing parts of your code easier

Type-along/demo where we discuss and experience aspects of (un)modular code development. **We will focus on the "why", not on the "how"**.

Image to get started: The curse of bad code design (Ten simple rules for quick and dirty scientific programming. PLoS Comput Biol 17(3): e1008549. https://doi.org/10.1371/journal.pcbi.1008549)

Slides

We also have some slides: https://github.com/coderefinery/modular-code-development. But here we will try to do this as collaborative type-along where instructors try to solve a problem together and where learners guide the instructors through a collaborative document.

Starting questions for the collaborative document

We share these questions in a common collaborative document and we wait until we have sufficiently many answers to question A. But we also encourage answering other questions which we revisit at the end of the demo.

```
A. What does "modular code development" mean for you?
B. What best practices can you recommend to arrive at well structured,
   modular code in your favourite programming language?
C. What do you know now about programming that you wish somebody told you earlier?
```

Additional questions

```
D. Do you design a new code project on paper before coding? Discuss pros
   and cons.
E. Do you build your code top-down (starting from the big picture) or bottom-up
   (starting from components)? Discuss pros and cons.
F. Would you prefer your code to be 2x slower if it was easier to read and understand?
```

Learning outcomes

- Know about **pure functions** (functions without side effects, functions which given same input always return same output).
- Learn why and how to **limit side effects** of functions.
- Discuss why and how to limit side effects of data. Also discuss when mutable data may be preferable.
- The Zen of Python
- Discuss why **single-purpose functions** are often preferred over multi-purpose functions.
- **Split-apply-combine**, which lets you more easily parallelize. Make your code modular in a way that lets you split the steps and parallelize.
- Think about **global vs local** data structures. It is not easy to separate them right.
- Understand how a command line interface to a code can improve usability and also make the code more versatile (to be combined with workflow management tools).
- Connect modular code development to the remaining lessons (version control, testing, documentation, reusability).

Our task

Data

The file temperatures.csv (raw csv file) contains hourly air temperature measurements for the observation station "Vantaa Helsinki-Vantaan lentoasema" (Helsinki airport) during 2022.

Origin of the data

Data obtained from https://en.ilmatieteenlaitos.fi/download-observations#!/ on 2023-09-27.

Data has been provided by the Finnish Meteorological Institute under the Creative Commons Attribution 4.0 International license (CC BY 4.0): https://en.ilmatieteenlaitos.fi/open-data-licence

Our initial goal

Our initial goal for this exercise is to plot a series of temperatures for **25 measurements** and to compute and plot the **arithmetic mean**. We imagine that we assemble a working script from various StackOverflow/ChatGPT recommendations and arrive at:

```python
import pandas as pd
from matplotlib import pyplot as plt

num_measurements = 25

# read data from file
data = pd.read_csv("temperatures.csv", nrows=num_measurements)
temperatures = data["Air temperature (degC)"]

# compute statistics
mean = sum(temperatures) / num_measurements

# plot results
plt.plot(temperatures, "r-")
plt.axhline(y=mean, color="b", linestyle="--")
plt.show()
plt.savefig("25.png")
plt.clf()
```

This example is in Python but we will try to see “through” the code and focus on the bigger picture and hopefully manage to imagine other languages in its place. For the Python experts: we will not see the most elegant Python.

Further goals

- Once we get this working for **25 measurements**, our task changes to also plot the **first 100** and the **first 500 measurements** in two additional plots.
- Then we wish to generalize the code so that a user can compute and plot this for **any number**, **without changing the code** (with a command line interface).

How we plan to solve it

Before we attempt to do this, we discuss with workshop participants how they would tackle this problem.

Together we improve the code based on suggestions from learners towards more modularity and re-usability.

Instructor note

Participants give suggestions and ask questions via collaborative document and instructor(s) try to follow and answer. They can also roughly follow the ideas and steps in the An iterative solution.

It is OK and good if mistakes happen and it is fun if the instructor(s) can convey a bit of “improv” feel to this lesson.

Additional exercises

Draw a call tree for one of your recent projects. Identify the functions in your call tree which are “pure” (which have no side-effects).

Types of encapsulation

Let’s say you want to move a code from one system to another. What are the things that can go wrong?

- **In-language dependencies**, e.g. Python. Can they all be expressed *only* in `requirements.txt`? Do you wrap everything in a container?
- **Paths**: Can you always use relative paths?
- **Operating-system (OS) dependencies, libraries, etc.**: Can you eliminate them?
- **Data files**: Are they controlled or few that you can migrate, or if you wanted to move it, are you forced to copy the whole directory without regard to what the file is (thus creating a lot of duplicates, and a big mess?)
- Are you using something that is **OS-specific** (GNU/Linux vs BSD)?
- Do you use support programs only available on certain computers? Fewer external utilities you use = easier portability.

What needs to be global vs what needs to be local?

- Global data can be “seen”/accessed in the entire code.
- Local data is only available in the local vicinity of its definition.
- Try to have as little global data as possible.
- Global data are often input parameters, configuration parameters, command-line arguments.
- But try to localize these to the “main” code/function.

An iterative solution

Before we start

We **don’t have to follow this line by line** but it’s important to study this example well before demonstrating this.

Emphasize that the example is Python but we will try to see “through” the code and **focus on the bigger picture** and hopefully manage to imagine other languages in its place.

We **collect ideas and feedback in the collaborative document while coding** and the instructor tries to react to that without going into the rabbit hole.

We recommend to go through this together where the instructor(s) demonstrate(s) and learners can commend, suggest, and ask questions, and we are either all in the same video room or everybody is watching via stream. In other words, for this lesson, **learners are not in separate breakout-rooms**.

Checklist

- Start with notebook
- Generalize from 1 figure to 3 figures
- Abstract code into functions
- From functions with side-effects towards stateless functions
- Move from notebook to script
- Initialize git
- Add `requirements.txt`
- Add test
- **Add command line interface**
- Split into multiple files/modules

Our initial version

We imagine that we assemble a working script from various StackOverflow recommendations and arrive at:

Python

```python
import pandas as pd
from matplotlib import pyplot as plt

num_measurements = 25

# read data from file
data = pd.read_csv("temperatures.csv", nrows=num_measurements)
temperatures = data["Air temperature (degC)"]

# compute statistics
mean = sum(temperatures) / num_measurements

# plot results
plt.plot(temperatures, "r-")
plt.axhline(y=mean, color="b", linestyle="--")
plt.show()
plt.savefig("25.png")
plt.clf()
```

- We test it out **in a notebook**.

We add axis labels

It's not the best placement but it works and later it will bite us (only the first plot will have labels) and we will improve it:

Python

```python
import pandas as pd
from matplotlib import pyplot as plt

plt.xlabel("measurements")
plt.ylabel("air temperature (deg C)")

num_measurements = 25

# read data from file
data = pd.read_csv("temperatures.csv", nrows=num_measurements)
temperatures = data["Air temperature (degC)"]

# compute statistics
mean = sum(temperatures) / num_measurements

# plot results
plt.plot(temperatures, "r-")
plt.axhline(y=mean, color="b", linestyle="--")
plt.show()
plt.savefig("25.png")
plt.clf()
```

Once we get this working for 25 measurements, our task changes to also plot the first 100 and the first 500 measurements in two additional plots.

Plotting also 100 and 500 measurements

- Next idea is perhaps code duplication.
- Then a for-loop to iterate over `[25, 100, 500]`:

Python

```python
import pandas as pd
from matplotlib import pyplot as plt

plt.xlabel("measurements")
plt.ylabel("air temperature (deg C)")

for num_measurements in [25, 100, 500]:

    # read data from file
    data = pd.read_csv("temperatures.csv", nrows=num_measurements)
    temperatures = data["Air temperature (degC)"]

    # compute statistics
    mean = sum(temperatures) / num_measurements

    # plot results
    plt.plot(temperatures, "r-")
    plt.axhline(y=mean, color="b", linestyle="--")
    plt.show()
    plt.savefig(f"{num_measurements}.png")
    plt.clf()
```

Abstracting the plotting part into a function

Python

```python
import pandas as pd
from matplotlib import pyplot as plt

plt.xlabel("measurements")
plt.ylabel("air temperature (deg C)")


def plot_temperatures(temperatures):
    plt.plot(temperatures, "r-")
    plt.axhline(y=mean, color="b", linestyle="--")
    plt.show()
    plt.savefig(f"{num_measurements}.png")
    plt.clf()


for num_measurements in [25, 100, 500]:

    # read data from file
    data = pd.read_csv("temperatures.csv", nrows=num_measurements)
    temperatures = data["Air temperature (degC)"]

    # compute statistics
    mean = sum(temperatures) / num_measurements

    # plot results
    #   plt.plot(temperatures, 'r-')
    #   plt.axhline(y=mean, color='b', linestyle='--')
    #   plt.show()
    #   plt.savefig(f'{num_measurements}.png')
    #   plt.clf()
    plot_temperatures(temperatures)
```

- Discuss what we expect before running it (some will expect this not to work because variables seem undefined).
- Then try it out (it actually works).
- Discuss problems with this solution (what if we copy-paste the function to a different file?).

The point of this step was that abstracting code into functions can be really good for reusability but just the fact that we created a function does not mean that the function is reusable since in this case it depends on a variable defined outside the function and hence there are side-effects.

Small improvements

- Abstracting into more functions.
- Notice how the comments got redundant:

Python

```python
import pandas as pd
from matplotlib import pyplot as plt


def plot_data(data, xlabel, ylabel):
    plt.plot(data, "r-")
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.axhline(y=mean, color="b", linestyle="--")
    plt.show()
    plt.savefig(f"{num_measurements}.png")
    plt.clf()


def compute_statistics(data):
    mean = sum(data) / num_measurements
    return mean


def read_data(file_name, column):
    data = pd.read_csv(file_name, nrows=num_measurements)
    return data[column]


for num_measurements in [25, 100, 500]:

    temperatures = read_data(
        file_name="temperatures.csv", column="Air temperature (degC)"
    )

    mean = compute_statistics(temperatures)

    plot_data(
        data=temperatures, xlabel="measurements", ylabel="air temperature (deg C)"
    )
```

Discuss what would happen if we copy-paste the functions to another project (these functions are stateful/time-dependent).

Emphasize how stateful functions and order of execution in Jupyter notebooks can produce unexpected results and explain why we motivate to rerun all cells before sharing the notebook.

Towards functions without side-effects

Improve to more stateless functions:

Python

```python
import pandas as pd
from matplotlib import pyplot as plt


def plot_data(data, mean, xlabel, ylabel, file_name):
    plt.plot(data, "r-")
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.axhline(y=mean, color="b", linestyle="--")
    plt.show()
    plt.savefig(file_name)
    plt.clf()


def compute_mean(data):
    mean = sum(data) / len(data)
    return mean


def read_data(file_name, nrows, column):
    data = pd.read_csv(file_name, nrows=nrows)
    return data[column]


for num_measurements in [25, 100, 500]:

    temperatures = read_data(
        file_name="temperatures.csv",
        nrows=num_measurements,
        column="Air temperature (degC)",
    )

    mean = compute_mean(temperatures)

    plot_data(
        data=temperatures,
        mean=mean,
        xlabel="measurements",
        ylabel="air temperature (deg C)",
        file_name=f"{num_measurements}.png",
    )
```

These functions can now be copy-pasted to a different notebook or project and they will still work.

Move from notebook to script

Adding unit tests is often the moment when notebook is not the right fit anymore.

But before we add tests:

- “File” -> “Save and Export Notebook As …” -> “Executable Script”
- `git init` and commit the working version.
- Add `requirements.txt` and motivate how that can be useful to have later.

As we continue from here, **create commits after meaningful changes** and later also share the repository with learners. This nicely connects to other lessons of the workshop.

Unit tests

Design code for testing.

- Move the main scope code into a main function.
- Discuss where to add a test and add a test to the statistics function:

Python

```python
import pandas as pd
from matplotlib import pyplot as plt
import pytest


def plot_data(data, mean, xlabel, ylabel, file_name):
    plt.plot(data, "r-")
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.axhline(y=mean, color="b", linestyle="--")
    #   plt.show()
    plt.savefig(file_name)
    plt.clf()


def compute_mean(data):
    mean = sum(data) / len(data)
    return mean


def test_compute_mean():
    result = compute_mean([1.0, 2.0, 3.0, 4.0])
    assert result == pytest.approx(2.5)


def read_data(file_name, nrows, column):
    data = pd.read_csv(file_name, nrows=nrows)
    return data[column]


def main():
    for num_measurements in [25, 100, 500]:

        temperatures = read_data(
            file_name="temperatures.csv",
            nrows=num_measurements,
            column="Air temperature (degC)",
        )

        mean = compute_mean(temperatures)

        plot_data(
            data=temperatures,
            mean=mean,
            xlabel="measurements",
            ylabel="air temperature (deg C)",
            file_name=f"{num_measurements}.png",
        )


if __name__ == "__main__":
    main()
```

Command-line interface

- Add a CLI for the input data file, the number of measurements, and the output file name.
- Example here is using click but it can equally well be optparse, argparse, docopt, or Typer.

- Discuss the motivations for adding a CLI:
  - We are able to modify the behavior without changing the code
  - We can run many of such scripts as part of a workflow

Python

```python
import pandas as pd
from matplotlib import pyplot as plt
import pytest
import click


def plot_data(data, mean, xlabel, ylabel, file_name):
    plt.plot(data, "r-")
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.axhline(y=mean, color="b", linestyle="--")
    plt.savefig(file_name)
    plt.clf()


def compute_mean(data):
    mean = sum(data) / len(data)
    return mean


def test_compute_mean():
    result = compute_mean([1.0, 2.0, 3.0, 4.0])
    assert result == pytest.approx(2.5)


def read_data(file_name, nrows, column):
    data = pd.read_csv(file_name, nrows=nrows)
    return data[column]


@click.command()
@click.option(
    "--num-measurements", required=True, type=int, help="Number of measurements."
)
@click.option("--in-file", required=True, help="File name where we read from.")
@click.option("--out-file", required=True, help="File name where we write to.")
def main(num_measurements, in_file, out_file):

    temperatures = read_data(
        file_name=in_file,
        nrows=num_measurements,
        column="Air temperature (degC)",
    )

    mean = compute_mean(temperatures)

    plot_data(
        data=temperatures,
        mean=mean,
        xlabel="measurements",
        ylabel="air temperature (deg C)",
        file_name=out_file,
    )


if __name__ == "__main__":
    main()
```

Split long script into modules

- Discuss how you would move some functions out and organize them into separate modules which can be imported to other projects: For instance `compute_mean` can be moved to `statistics.py`.
- Discuss naming.
- Discuss interface design.

Summarize in the collaborative document

- Now return to initial questions on the collaborative document and discuss questions and comments. If there is time left, there are additional questions and exercises.
- It is easier and more fun to teach this as a pair with somebody else where one person can type and the other person helps watching the questions and commends and relays them to the co-instructor.